

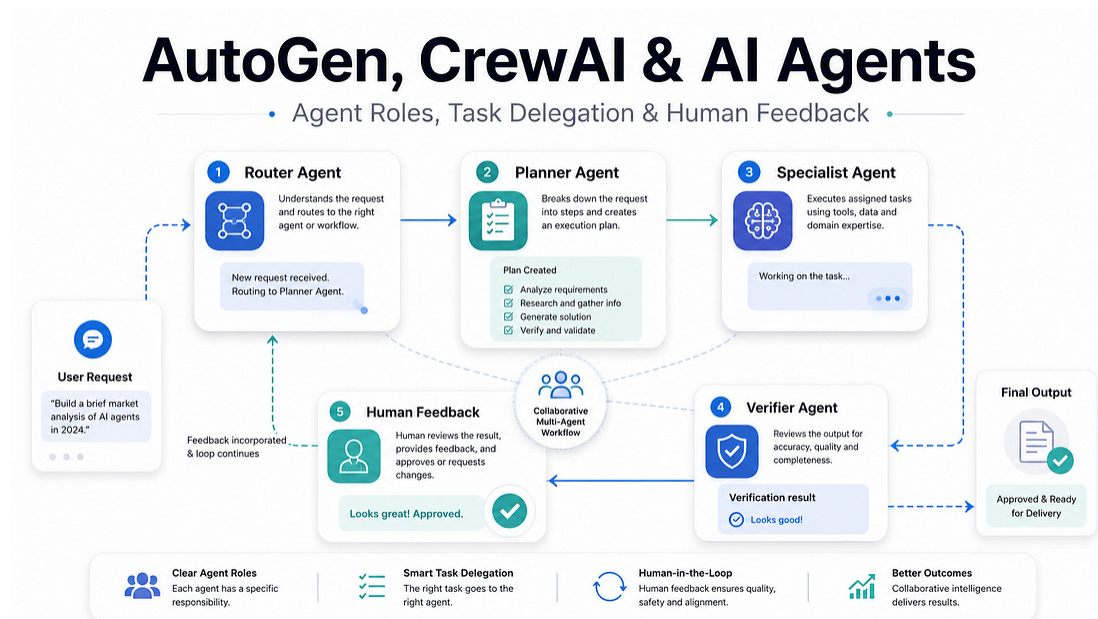
AutoGen, CrewAI & Multi-Agent Orchestration

Agent roles, task delegation, multi-agent orchestration, human feedback integration, guardrails, evaluation, and production readiness.



AI ENGINEERING INSIDER

MAY 27, 2026



1. What Interviewers Are Testing

They are not only asking, “Do you know CrewAI or AutoGen?” They are usually testing whether you can design a **reliable agentic workflow** where multiple LLM-powered agents collaborate without becoming expensive, slow, unsafe, or unpredictable.

This Substack is reader-supported. To receive new posts and support my work,

consider becoming a free or paid subscriber.

✓ **Subscribed**

The key skill is explaining when to use:

- ↳ **Single agent** for simple tool-use workflows
- ↳ **Planner-executor** for multi-step decomposition
- ↳ **Supervisor-worker** for controlled delegation
- ↳ **Crew/team-based agents** for role-specialized collaboration
- ↳ **Human-in-the-loop** for approvals, ambiguity, policy risk, or irreversible actions
- ↳ **Verifier/critic agents** for quality checks before output or tool execution

AutoGen remains important for understanding multi-agent conversation patterns, but Microsoft's GitHub now marks AutoGen as in maintenance mode, and Microsoft provides migration guidance toward the Microsoft Agent Framework for newer enterprise agent work. ([GitHub](#)) CrewAI is currently framed around agents, tasks, crews, flows, guardrails, memory, knowledge, and observability.

2. Core Concept Cheat Sheet

Agent

An **agent** is an LLM-driven unit with instructions, tools, memory/state, and a role-specific objective. In CrewAI, an agent can perform tasks, make decisions based on role and goal, use tools, collaborate with other agents, maintain memory, and delegate when allowed. ([CrewAI Documentation](#))

Interview answer pattern:

“I treat an agent as a bounded worker, not a magical autonomous employee. I define its role, tools, permissions, input contract, output schema, termination condition, and failure policy.”

Task

A **task** is a concrete assignment given to an agent. In CrewAI, tasks define execution details, including description, responsible agent, required tools, and collaboration behavior. ([CrewAI Documentation](#))

Good task definition:

- ↳ Clear objective
 - ↳ Input context
 - ↳ Allowed tools
 - ↳ Expected output format
 - ↳ Success criteria
 - ↳ Guardrail validation
 - ↳ Retry/escalation behavior
-

Crew / Team

A **crew** is a group of agents collaborating toward a goal. CrewAI defines a crew as a collaborative group of agents that execute tasks according to a workflow strategy. ([CrewAI Documentation](#))

Example:

- ↳ Research Agent
- ↳ Data Extraction Agent

- ↳ SQL Agent
 - ↳ Compliance Reviewer
 - ↳ Final Answer Synthesizer
-

Flow / Workflow

A **flow** controls state, execution order, branching, and event-driven orchestration. CrewAI describes Flows as structured, event-driven workflows that manage state and control execution; Crews are the units of work inside those Flows. ([CrewAI Documentation](#))

Senior-level point:

“Agents should not own the entire control plane. The workflow engine should own state, retries, approvals, branching, and auditability.”

Handoff

A **handoff** transfers control from one agent to another specialist. OpenAI’s Agents SDK describes handoffs as a way for an agent to delegate tasks to another agent, especially when different agents specialize in distinct areas. ([OpenAI GitHub](#))

Example:

- ↳ Triage Agent receives user request
 - ↳ Detects refund issue
 - ↳ Hands off to the Refund Agent
 - ↳ Refund Agent uses policy and order tools
 - ↳ Human approval required before refund execution
-

Human-in-the-Loop

Human feedback is used when an agent needs clarification, approval, review, correction, or escalation. CrewAI documents human input during execution as useful when agents require additional information or clarification for complex decision-making. ([CrewAI Documentation](#)) LlamaIndex similarly supports human-in-the-loop through workflow events such as input-required and human-response events. ([Developer Documentation](#))

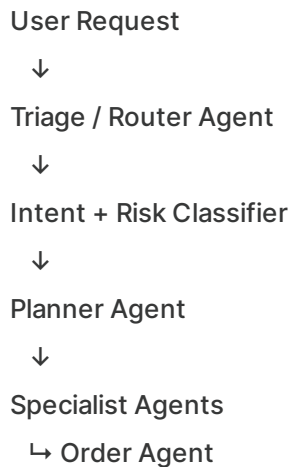
Common triggers:

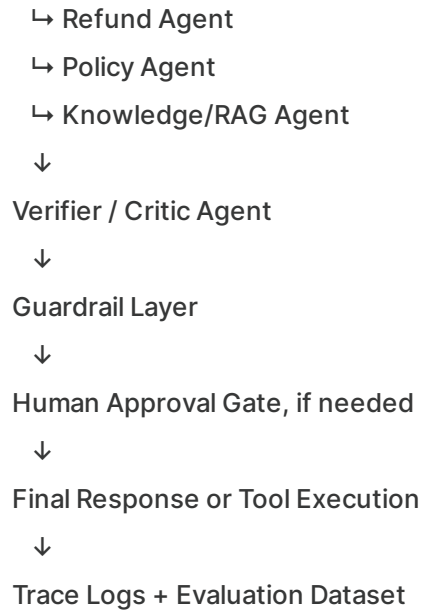
- ↳ Low confidence
 - ↳ High financial impact
 - ↳ Legal/compliance-sensitive action
 - ↳ Ambiguous user intent
 - ↳ Tool action is irreversible
 - ↳ Agent output fails validation
-

3. Framework Comparison for Interviews

Framework	Best Interview Framing	Strong Use Case
AutoGen	Multi-agent conversation, Assistant/UserProxy patterns, agent collaboration	Research prototypes, conversational agent teams, code-oriented workflows
CrewAI	Role-based agents, tasks, crews, flows, hierarchical delegation	Business workflow automation, content/research/reporting agents, role-specialized teams
LangGraph	State-machine agent orchestration, graph-based control, supervisor patterns	Production-grade agent workflows needing explicit state and branching
OpenAI Agents SDK	Agents, tools, handoffs, guardrails, tracing, human review	OpenAI-native production agent workflows
LlamaIndex AgentWorkflow	Event-driven multi-agent workflows, RAG-heavy agent systems	Agentic RAG, workflow-based retrieval, human approval events
Microsoft Agent Framework	Enterprise successor path for Microsoft ecosystem	Azure, .NET/Python enterprise agents, Microsoft stack integration

4. Reference Architecture: Multi-Agent Customer Support Workflow





Strong interview explanation:

“I would separate orchestration from reasoning. The workflow engine controls state, routing, retries, approvals, and observability. Agents perform bounded reasoning or tool use. Every agent has a role, tool scope, output schema, and evaluation criteria. High-risk actions go through human approval before execution.”

OpenAI's Agents SDK also emphasizes orchestration, handoffs, guardrails, human review, results/state, and observability as workflows become more complex. ([OpenAI Developers](#)) It's tracing records of LLM generations, tool calls, handoffs, guardrails, and custom events during agent runs. ([OpenAI GitHub](#))

5. Mock Interview Q&A

Q1. What is the difference between an agent and a workflow?

Answer:

An agent is a reasoning component that uses an LLM, instructions, tools, and context to complete a bounded task. A workflow is the control layer that decides execution order, branching, retries, approvals, and state transitions.

In production, I avoid letting the agent control everything. I let the workflow own deterministic control and use agents only where reasoning, language understanding, planning, or tool selection is needed.

Strong signal:

“Agents reason. Workflows govern.”

Q2. When would you use multiple agents instead of one agent?

Answer:

I use multiple agents when the task benefits from specialization, separation of responsibility, or independent verification.

Examples:

- ↳ A **Research Agent** gathers evidence
- ↳ A **SQL Agent** queries structured data
- ↳ A **Policy Agent** checks compliance
- ↳ A **Verifier Agent** validates the answer
- ↳ A **Human Reviewer** approves risky actions

I would not use multiple agents just because the framework supports it. Multi-agent systems increase latency, cost, debugging complexity, and

failure surface.

Q3. How would you design agent roles?

Answer:

I define each role using five constraints:

- ↳ **Responsibility:** what the agent owns
- ↳ **Tool access:** what the agent can call
- ↳ **Input contract:** what context does it receive
- ↳ **Output schema:** what it must return
- ↳ **Failure policy:** retry, fallback, or escalate

For example, a Refund Agent should not have access to all customer data. It should receive only the order context, refund policy, transaction status, and allowed refund action.

Q4. Explain CrewAI's core abstraction.

Answer:

CrewAI is built around agents, tasks, crews, and flows. Agents are autonomous units with roles and goals. Tasks are assignments completed by agents. Crews are collaborative groups of agents. Flows provide structured, event-driven control over state and execution. ([CrewAI Documentation](#))

In an interview, I would describe CrewAI as a strong fit for role-based business workflows where you want agents to collaborate like a team, but still need workflow-level control.

Q5. What is hierarchical delegation in CrewAI?

Answer:

Hierarchical delegation means a manager agent coordinates worker agents. CrewAI's hierarchical process introduces a manager-style structure where a manager agent coordinates the workflow, delegates tasks, and validates outcomes. ([CrewAI Documentation](#))

I would use this for workflows where the order of work depends on intermediate findings. For example, in market research, a manager agent could assign one worker to collect competitor data, another to summarize pricing, and another to validate sources.

Q6. What are the risks of hierarchical multi-agent systems?

Answer:

The main risks are:

- ↳ The manager agent delegates incorrectly
- ↳ Workers duplicate effort
- ↳ Context becomes inconsistent across agents
- ↳ Token cost grows quickly
- ↳ Agents agree with each other without real verification
- ↳ The system becomes hard to debug

I mitigate this with structured outputs, shared state, trace IDs, deterministic routing where possible, and evaluator checks after each major step.

Q7. What is AutoGen best known for?

Answer:

AutoGen is known for its multi-agent conversation patterns, in which agents can communicate and cooperate to solve tasks. AutoGen AgentChat is a high-level API for building conversational single-agent and multi-agent applications, while AutoGen Core provides a lower-level event-driven model for more flexible multi-agent systems. ([Microsoft GitHub](#))

For interviews, I would explain AutoGen as historically important for agent-to-agent conversations, code execution patterns, and research-style multi-agent collaboration.

Q8. Would you choose AutoGen for a new production system today?

Answer:

I would be careful. I would still discuss AutoGen because it is important historically and conceptually, but for a new production system, I would evaluate CrewAI, LangGraph, OpenAI Agents SDK, LlamaIndex Workflows, or Microsoft Agent Framework, depending on the stack.

The reason is that Microsoft's AutoGen GitHub currently says AutoGen is in maintenance mode and will not receive new features or enhancements. ([GitHub](#)) Microsoft also provides migration documentation from AutoGen to Microsoft Agent Framework. ([Microsoft Learn](#))

Q9. What is the difference between handoff and tool calling?

Answer:

Tool calling means an agent invokes a function, an API, a database query, a browser action, or a retrieval tool.

Handoff means one agent transfers responsibility to another agent. In OpenAI's Agents SDK, handoffs allow an agent to delegate tasks to another specialist agent, and the handoff is represented as a tool to the LLM. ([OpenAI GitHub](#))

Example:

↳ Tool call: `get_order_status(order_id)`

↳ Handoff: Triage Agent transfers conversation to Refund Agent

Q10. How do you prevent agents from hallucinating tool results?

Answer:

I prevent this by forcing tools to return structured data and requiring the agent to cite or reference tool outputs directly. I also separate tool execution from final answer generation.

My pattern:

↳ Tool returns JSON

↳ The agent must summarize only the fields present in JSON

↳ Verifier checks the answer against the tool output

↳ Missing data must be reported as unknown

↳ High-risk or low-confidence answers escalate

Q11. How do you integrate human feedback?

Answer:

I use human feedback at three layers:

- ↳ **Pre-execution approval** for risky actions
- ↳ **Clarification loop** when intent is ambiguous
- ↳ **Post-output review** for quality improvement

For example, before issuing a refund, the agent prepares a proposed action:

```
{  
  "action": "issue_refund",  
  "amount": 149.99,  
  "reason": "duplicate charge",  
  "confidence": 0.82,  
  "requires_human_approval": true  
}
```

The workflow pauses until a human approves, rejects, or edits the action.

Q12. When should a human approval gate trigger?

Answer:

A human approval gate should trigger when:

- ↳ The action is irreversible
- ↳ The financial amount exceeds a threshold
- ↳ The user intent is ambiguous
- ↳ The agent's confidence is low
- ↳ Policy conflict is detected
- ↳ The output involves legal, medical, HR, security, or compliance risk
- ↳ The tool has write access

This keeps autonomy bounded and auditable.

Q13. How would you evaluate a multi-agent system?

Answer:

I would evaluate it at multiple levels:

LayerMetricTask levelTask completion rate, success/failure rateAgent levelRole adherence, tool accuracy, schema complianceWorkflow levelCorrect routing, retry success, handoff accuracyOutput levelFactuality, groundedness, helpfulnessSafety levelPolicy violations, unsafe tool callsOps levelLatency, cost, token usage, failure rate

For agentic systems, I care not only about the final answer but also the **trajectory**: which agents were called, which tools were used, whether the delegation was correct, and whether unnecessary steps increased cost.

Q14. What is trajectory evaluation?

Answer:

Trajectory evaluation checks the agent's path, not just the final answer.

For example:

- ↳ Did the router choose the correct specialist?
- ↳ Did the planner correctly decompose the task?
- ↳ Did the SQL agent call the right query?
- ↳ Did the verifier catch missing evidence?
- ↳ Did the workflow ask for human approval when required?

This is critical because two systems can produce similar final answers, but one may have used unsafe, expensive, or unreliable steps.

Q15. What guardrails would you add to a CrewAI or AutoGen-style system?

Answer:

I would add guardrails at four points:

- ↳ **Input guardrails:** detect prompt injection, unsafe intent, missing data
- ↳ **Tool guardrails:** restrict tools by role and permission
- ↳ **Output guardrails:** validate schema, citations, policy compliance
- ↳ **Action guardrails:** require approval before write operations

CrewAI supports task guardrails to validate task outputs before acceptance. ([CrewAI Documentation](#)) OpenAI's Agents SDK also treats guardrails and human review as part of more complex agent workflows. ([OpenAI Developers](#))

Q16. How do you design memory for multi-agent systems?

Answer:

I separate memory into:

- ↳ **Working memory:** current task state
- ↳ **Episodic memory:** previous interactions or workflow events
- ↳ **Semantic memory:** domain knowledge, documentation, policies
- ↳ **Operational memory:** traces, errors, approvals, tool results

I avoid giving every agent full memory access. Each agent receives only the context needed for its role. This reduces leakage, prompt bloat, and cross-agent confusion.

Q17. How would you debug a failed multi-agent workflow?

Answer:

I would inspect the trace step by step:

- ↳ User input
- ↳ Router decision
- ↳ Planner output
- ↳ Agent selected
- ↳ Tool calls
- ↳ Intermediate outputs
- ↳ Guardrail failures
- ↳ Handoff path
- ↳ Final answer

OpenAI's Agents SDK tracing captures LLM generations, tool calls, handoffs, guardrails, and custom events, which is exactly the kind of evidence needed for debugging agent workflows. ([OpenAI GitHub](#))

Q18. What is the biggest anti-pattern in multi-agent design?

Answer:

The biggest anti-pattern is creating too many agents without clear ownership.

Bad design:

Research Agent
Analysis Agent
Thinking Agent
Reasoning Agent
Decision Agent
Final Agent

This creates overlapping responsibilities and noisy delegation.

Better design:

Router Agent
Policy Retrieval Agent
Transaction Tool Agent
Verifier Agent
Human Approval Gate

Each component has a specific role, contract, and measurable success criteria.

Q19. How do you control cost in multi-agent systems?

Answer:

I control cost by:

- ↳ Using deterministic routing before LLM routing
- ↳ Keeping agents' role-specific and context-limited
- ↳ Caching, retrieval, and tool results
- ↳ Using smaller models for routing/classification
- ↳ Calling verifier agents only for high-risk cases
- ↳ Limiting max delegation depth
- ↳ Adding early stopping conditions
- ↳ Tracking token usage per agent and per task

A multi-agent architecture can easily multiply the cost if every step calls a large model.

Q20. How would you explain agent delegation to a non-technical stakeholder?

Answer:

I would say:

“Instead of one AI assistant trying to do everything, we create a controlled team of specialized assistants. One understands the request, one retrieves the right information, one performs the task, and one checks the result. For sensitive actions, a human approves before anything is executed.”

This explanation makes the value clear without overusing technical terms.

6. Senior-Level System Design Prompt

Prompt

Design a multi-agent system using CrewAI, AutoGen-style concepts, or another agent framework for an enterprise customer support assistant. The system must handle order lookup, refund eligibility, policy Q&A, escalation, and human approval for high-risk actions.

Strong Answer Outline

Architecture

- ↳ API Gateway receives request
- ↳ Auth layer validates user/session
- ↳ Router Agent classifies intent
- ↳ Planner Agent decomposes task
- ↳ Specialist agents execute bounded tasks
- ↳ Retrieval Agent queries policy knowledge base
- ↳ Tool Agent calls order/refund APIs
- ↳ Verifier Agent checks correctness
- ↳ Human approval gate handles risky actions
- ↳ Final Response Agent generates customer-facing response
- ↳ Observability layer logs traces, token usage, decisions, and approvals

Agent Roles

Agent	Responsibility	Tools
Router Agent	Intent classification and risk level	Classifier, policy tags
Planner Agent	Task decomposition	None or limited planning tools
Policy Agent	Retrieve refund/shipping/account policy	Vector DB, search
Order Agent	Read order status	Order API
Refund Agent	Propose refund action	Refund API, but write access gated
Verifier Agent	Validate output against evidence	Tool outputs, policy snippets
Human Reviewer	Approve/reject risky action	Admin UI

Human Feedback Integration

- ↳ Refunds over the threshold require approval
- ↳ Policy conflict triggers escalation
- ↳ Low confidence triggers clarification
- ↳ Human corrections are stored as evaluation examples
- ↳ Repeated failure cases update prompts, routing rules, or retrieval data

Failure Handling

Failure	Mitigation
Wrong agent selected	Router evaluation dataset, fallback router
Tool timeout	Retry with backoff, user-safe message
Conflicting policy	Escalate to human
Low confidence	Ask clarification
Invalid output schema	Retry with structured output correction
Prompt injection	Input filtering and tool permission boundaries
Cost spike	Max depth, token budgets, cached retrieval

7. Framework Selection Answer

A strong interview response:

“For a quick role-based prototype, I might use CrewAI because its agent/task/crew abstractions map well to business workflows. For production systems requiring explicit state, branching, retries, and deterministic control, I would consider LangGraph or LlamaIndex Workflows. For OpenAI-native apps, I would evaluate the OpenAI Agents SDK because it provides agents, handoffs, guardrails, human review, and tracing. For existing AutoGen systems, I would maintain or migrate carefully, but for new Microsoft enterprise builds I would evaluate Microsoft Agent Framework because AutoGen is now in maintenance mode.”

8. Practical Interview

Use these in interviews:

- ↳ “I don’t start with agents. I start with the workflow and add agents only where reasoning is required.”
- ↳ “Every agent needs a role, tool boundary, output contract, and failure policy.”
- ↳ “Human-in-the-loop is not a weakness. It is a control mechanism for high-risk autonomy.”
- ↳ “Multi-agent systems should be evaluated by trajectory, not only final answer quality.”

↳ “The orchestrator should own the state. Agents should own bounded reasoning tasks.”

↳ “Delegation without observability becomes distributed hallucination.”

↳ “A verifier agent is useful, but it must check evidence, not just critique style.”

↳ “For production, I care more about traceability, permissions, and rollback than demo autonomy.”

9. Final Mock Interview Challenge

Question

You built a CrewAI-style multi-agent system with a manager agent, a researcher agent, a SQL agent, and a writer agent. Users complain that answers are inconsistent and sometimes cite data that the SQL tool does not return. What do you do?

Best Answer

I would debug the workflow at the trajectory level.

First, I would inspect the traces to see whether the SQL agent actually queried the correct data and whether the writer agent received the raw SQL results or only a summary. If the writer is inventing values, I would change the contract so the writer can only use structured SQL output fields.

Second, I would add schema validation between agents. The SQL agent should return a typed object, not prose. The writer agent should receive that object and produce an answer with the required field references.

Third, I would add a verifier agent that compares the final answer against the SQL output. If the answer includes numbers not found in the SQL result, the workflow fails and retries or escalates.

Fourth, I would reduce unnecessary delegation. If the manager agent is routing inconsistently, I would replace part of the routing with deterministic rules.

Finally, I would add regression tests using historical user questions and expected SQL-backed answers. I would track groundedness, tool accuracy, routing accuracy, latency, and cost per successful task.

Strong closing line:

“The fix is not just better prompting. The fix is stricter contracts between agents, evidence-bound generation, trace-based debugging, and workflow-level guardrails.”

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed